28 oa 2C

Modelo de programación funcional.

El modelo de programación funcional es un paradigma declarativo que se enfoca en resolver problema mediante el uso de funciones matemáticas puras y datos inmutables, en lugar de cambiar estados o dar instrucciones paso a paso.

En este estilo de programación las funciones son ciudadanas de primera clase, porque sus expresiones pueden ser asignadas a variables como se haría con cualquier otro valor; además de que pueden crearse funciones de orden superior.

La programación funcional tiene sus raíces en el cálculo lambda, un sistema formal desarrollo en los años 1930 para investigar la naturaleza de las funciones, la naturaleza de la computabilidad y su relación con la recursión. Los lenguajes funcionales priorizan el uso de recursividad y aplicación de funciones de orden superior para resolver problemas que en otros lenguajes se resolverían mediante estructuras de control (por ejemplo, ciclos).

Algunos lenguajes funcionales tambien buscan eliminar la mutabilidad o efectos secundarios; en contraste con la programación imperativa, que se basa en los cambios de estado mediante la mutación de variables.

Características Principales.

- **Funciones Puras:** Con el mismo valor de entrada, siempre devuelven exactamente el mismo valor de salida, sin alterar nada fuera de ellas.
- **Inmutabilidad:** los datos no se pueden modificar una vez creados; si se necesita un cambio, se genera un nuevo dato o estructura.
- **Sin efectos secundarios:** las funciones evitan modificar variables globales, bases de datos o pantallas de manera oculta.
- **Enfoque declarativo:** Describes qué quieres lograr y no cómo hacerlo detalladamente con bucles o controles de flujo manuales.
- **Funciones de orden superior:** las funciones pueden recibir otras funciones como parámetros o devolverlas como resultado

Ventajas

- **Código más seguro:**
Al no cambiar estados compartidos, hay menores errores inesperados y se facilita trabajar con varios procesos a la vez (concurrencia).
- **Fácil de probar:**
Las funciones puras son muy sencillas de probar de forma aislada.
- **Mayor legibilidad:**
El código se vuelve modular, claro y parecido a la lógica matemática.

CALCULO LAMBDA DE ALONZO CHURCH

El cálculo lambda es un sistema formal desarrollado por el matemático Alonzo Church en la década de 1930 para investigar la definición de función, la aplicación y la recursión.

Se puede considerar al cálculo lambda como uno de los lenguajes universales de programación más minimalistas. Se le considera el lenguaje de programación funcional más pequeño del mundo porque no usa números, cadenas ni booleanos, sino exclusivamente funciones. Se compone de tres elementos:

- Variable

Un símbolo o nombre que representa un valor o entrada (ej. x).

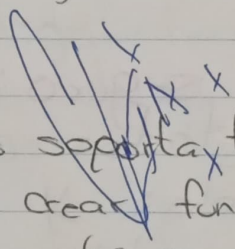
- Abstracción

La creación de una función anónima, escrita con una letra épsilon o lambda λ, un parámetro, un punto y un cuerpo (ej. λ.x.x+1).

- Aplicación

El acto de aplicar una función a un argumento o entrada (ej (λx.x+1)2)

Parámetros múltiples



Parece que el cálculo lambda solo soporta funciones con un solo parámetros, pero podemos crear funciones que toman 2 parámetros o más con una técnica llamada currificación.

- $(\lambda x.\lambda y.\lambda z.xyz)$ equivale a $f(x, y, z) = ((x\ y)\ z)$

A menudo se escribe directamente λxy.<cuerpo> en lugar de λx.λy.<cuerpo>

El cálculo lambda es universal porque cualquier función computable puede ser expresada y evaluada a través de él. Por lo tanto, es equivalente a las máquinas de Turing. Sin embargo, el cálculo lambda no hace énfasis en el uso de las reglas de transformación y no considera las máquinas reales que pueden implementarlo.

Fue uno de los muchos sistemas relacionados que se propusieron a finales de la década de 1920 y durante la de 1930. Fue una época apasionante para la informática teórica, aunque todavía no se habían inventado las computadoras.
Muchos grandes matemáticos, como Alonzo Church, Kurt Gödel, Alan Turing y Emil Post, intentaban comprender la noción de computación.

Valores

Se dice que un término λ está en forma normal si no hay reducciones β que se apliquen; es decir, si no hay subtérminos de la forma $(\lambda x.M)N$. Por ejemplo, los términos xy y $\lambda x.y$ están en forma normal. Los términos en forma normal corresponden a valores en OCaml. son objetos que no se pueden evaluar más.

Se dice que un término es normalizable si existe una secuencia de reducciones α y β que conducen a una forma normal. Debido a la propiedad de Church-Rosser, si un término es normalizable, entonces su forma normal es única (salvo conversación α).
Una secuencia de reducciones que conduce a una forma normal se corresponde aproximadamente con un cálculo de parada en Ocmal.

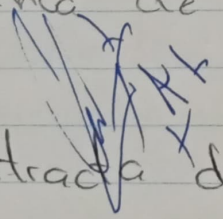
06 10 26

Elixir y Erlang.

Erlang y Elixir son dos lenguajes de programación estrechamente relacionados que están diseñados para crear sistemas distribuidos, altamente concurrentes, tolerantes a fallos y de tiempo real.

Erlang

Creado por la empresa de telecomunicaciones Ericsson a finales de los 80 (y lanzado en 1998) para gestionar centrales telefónicas que no podían permitirse jamás dejar de funcionar.
Es un lenguaje de programación funcional, concurrente y con tipado dinámico.
Corre sobre BEAM (Máquina virtual de Erlang), la cual permite manejar millones de procesos ligeros de manera simultánea.
Utiliza la filosofía "dejar que falle" (let it crash): si un proceso falla, otro supervisor lo reinicia sin tumbar todo el sistema.
Se usa en sistemas de mensajería como WhatsApp, telecomunicaciones, servidores de chat y aplicaciones con disponibilidad continua de "9 nueves".



BEAM: Máquina Abstracta de Erlang de Bogdan.

Se utilizan exclusivamente en el backend.

Elixir

Creado en 2011 por José Valim con el objetivo de tener una sintaxis moderna, amigable y extensible aprovechando toda la potencia de la máquina virtual de Erlang.

Relación con Erlang: Elixir está construido sobre Erlang. Todo el código de elixir se compila a "bytecode" que ejecuta la máquina virtual BEAM, lo que significa que un programa en Elixir puede usar bibliotecas de Erlang sin ningún problema.

Sintaxis y herramienta
es mucho más legible para desarrolladores acostumbrados a lenguajes modernos (inspirado en Ruby), e incluye metaprogramación con macros y herramientas muy productivas (como el framework web Phoenix).

USOS: Desarrollo web de alto rendimiento, sistemas en tiempo real, APIs y procesamiento distribuido de datos.

Se usa en WhatsApp ejecutandose sobre el sistema operativo FreeBSD como el núcleo de su infraestructura de mensajería, lo que le permite manejar más de 2 millones de conexiones simultáneas por servidor y procesar más de 100 mil millones de mensajes diarios con un equipo técnico reducido.

13 Oct 26

Funciones de orden superior (map, filter, fold/reduce) sobre listas.

Las funciones de orden superior son aquellas que pueden recibir una función como argumento o devolver una función como resultado.
Sobre listas (colecciones), map, filter y fold/reduce son las herramientas básicas de la programación declarativa para procesar datos sin usar bucles explícitos (for o while).

Map (Transformar)

El uso de map aplica una determinada función a todos los elementos de una entrada o lista.
Esta es su forma:

Forma

```
map(funcion_a_aplicar, lista_de_entradas)
```

Ejemplo

```
lista = [1, 2, 3, 4, 5]
al_cuadrado = []
for i in lista:
    al_cuadrado.append(i**2)
```

Sin embargo, existe una forma más fácil de hacerlo con map. Más sencilla y corta:

```
lista = [1,2,3,4,5]
al_cuadrado: list(map(lambda x: x**2, lista))
```

La mayoría de las veces map es usado conjuntamente con funciones lambda.

Filter (Filtrar)

La función Filter devuelve un nuevo array con los elementos que cumplan con la condición de la función callback. Como su nombre indica, filter crea una lista de elementos si usados en la llamada a una función True. Es decir, filtra los elementos de una lista usando un determinado criterio.

Por ejemplo, el siguiente código devuelve un nuevo array con los números pares del array original

```
lista = rang 5 5
menor_cero = list (filter (lambda x: x<0, lista))
  Print (menor_cero)
    # salida: [-5, -4, -3, -2, -1]
```

La función filter es similar a un bucle, y de hecho podríamos conseguir lo mismo con un bucle y un if, pero su uso es más rápido.

Ejemplo en Haskell

```
main::IO ()
main = print (filter even [1,2,3,4,5,6]
```

Resultado

[2, 4, 6]

fold / reduce

La función reduce es una función de orden superior que permite procesar los elementos de una lista para combinarlos y obtener un resultado final. Se utiliza en programación funcional.

Como funciona

reduce toma tres elementos principales

1. Una función: Indica qué operación se realiza entre los elementos.
2. Un valor inicial: es el valor con el que comienza la operación.
3. Una lista: Contiene los elementos que se van a procesar.

La función va combinando los elementos y actualizando un acumulador hasta obtener el resultado final.

Código en haskell

```
main :: IO ()
main = print (foldl (+) 0 [1,2,3,4])
```

Resultado

10.

26 10 26

Cuadro Comparativo

Característica	Prefija	Infija	Postfija
Definición	La función se escribe antes de sus argumentos.	El operador se escribe entre sus operandos.	El operador se escribe después de sus operandos.
Sintaxis	función argumento1 argumento2	argumento1 operador argumento2	argumento1 argumento2 operador.
Ejemplo en haskell	(+) 4 2	4 + 2	4 2 +
Uso en haskell	Es la forma habitual de llamar funciones	Es habitual para operadores aritméticos y lógicos	No se admite directamente como expresión aritmética estándar.
Paréntesis	los operadores se escriben entre paréntesis para usarlos como funciones. ej (+)	No se requiere paréntesis, para controlar la prioridad.	No se necesita paréntesis si se sigue una notación postfija bien definida.
Ventajas	Facilita la aplicación de funciones y la programación funcional.	Permite escribir operaciones de manera natural y legible	Facilita la evaluación mediante una pila.
Desventajas	Puede resultar menos intuitiva para operaciones matemáticas.	Puede requerir paréntesis para evitar ambigüedades	Requiere implementar un evaluador para utilizarla en haskell.

03 11 26

Intervalos, Operadores y evaluación perezosa en haskell

Intervalos

Los intervalos permiten generar listas de elementos secuenciales de forma rápida. Se apoya en clase de tipos Enum.

Sintaxis básica: [inicio...fin] genera una secuencia inclusiva.

- Con paso (step): [inicio, siguiente..fin] determina el incremento restando el segundo elemento del primero.

Ejemplo:

Lista del 1 al 10

```
[1..10]          --[1,2,3,4,5,6,7,8,9,10]
```

Letras de la 'a' a la 'e'

```
['a'..'e']       --"abcde"
```

Operadores

En haskell, los operadores son simplemente funciones infijas (se colocan entre sus argumentos).

Propiedades clave:

- Infijo a prefijo: Puedes usar cualquier operador como una función normal envolviendolo entre paréntesis: (+) 2 3.
- Prefijo a infijo: Puedes usar una función de dos argumentos como operador infijo envolviéndola en acentos graves (backticks): 5 `mod` 2.
- Precedencia y Asociatividad: se definen con infixl (asociativo a la izquierda), infixr (derecha) o infix (no asociativo) seguido de un nivel de prioridad del 0 al 9.

Operadores Comunes y Cruciales.

- : (cons) - Agrega un elemento al principio de una lista: 1:[2,3] resulta en [1,2,3].
- ++ - Concatena dos listas: [1,2] ++ [3,4].
- . - Composición de funciones: (f . g) x es equivalente a f (g x).
- $ - Aplicación de funciones con baja prioridad. Evita el exceso

parentesis: f (g x) se convierte en f $ g x.

aluación Perezosa. (Lazy Evaluation)

skell no evalúa los expresiones cuando se definen
cuando su resultado es estrictamente necesario para
el programa continúe (llamado por necesidad).

ntajos.
structuras de datos infinitas: Puede crear intervalo
fin porque haskell solo generará los elementos q
pidos.
rtocircuito automático: si una condición ya es falsa
evalúa el resto.
odularidad: Separa la generación de datos del
ntrol de flujo.

skell
l segundo elemento (1 'div' 0) nunca se evalúa
in = print (length [4, 1 'div' 0, 8])
Resultado: 3

jemplo corto para entender la evaluación perezosa es
ar la longitud de una lista que contiene una operación
mática inválida (como una división por cero).